



DOCUMENTATION TECHNIQUE

# Présentation du projet

Stack, démarrage, structure et fonctionnalités livrées

Plateforme digitale premium de la marque personnelle **Romial Kenmogne** : *Business & Financial Consultant | Project Manager* — Europe · Afrique · International.

**Understand Money. Build Businesses. Invest. Create Wealth.**

Le site est trilingue (**FR / DE / EN**) dès le lancement, dispose d'un mode clair/sombre, d'un CMS complet pour le propriétaire, d'un hub éditorial (RK Insights), d'un catalogue de livres, d'un formulaire de contact qualifié et d'une base SEO/accessibilité conforme aux exigences du cahier des charges 2026.

# Sommaire

---

1. Stack technique et justification

---

2. Démarrage rapide (développement local)

---

3. Scripts npm

---

4. Structure du projet

---

5. Multilingue

---

6. Thème clair / sombre

---

7. Supervision et données personnelles

---

8. Publication, aperçu et mise en cache

---

9. Sécurité

---

10. Documentation

---

11. État de la livraison

---

# 1. Stack technique et justification

| Brique          | Choix                                                                    | Pourquoi                                                                                                                                                                                                                          |
|-----------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework       | <b>Next.js 16</b> (App Router, React 19, TypeScript strict)              | Rendu statique/serveur pour le SEO, routes localisées natives, écosystème mature, gratuit                                                                                                                                         |
| CMS             | <b>Payload CMS 3</b> embarqué dans l'application ( <code>/admin</code> ) | CMS headless <i>dans</i> le même projet → un seul déploiement, une seule base ; localisation par champ FR/DE/EN, brouillons, programmation, versions, rôles, médiathèque. Open source, auto-hébergé, <b>aucun coût de licence</b> |
| Base de données | <b>PostgreSQL 16</b>                                                     | Relationnel (articles ↔ expertises ↔ projets ↔ livres), sauvegardes standard                                                                                                                                                      |
| Styles          | <b>Tailwind CSS v4</b> + design tokens CSS sémantiques                   | Thème clair/sombre sans couleur arbitraire, cohérence garantie                                                                                                                                                                    |
| i18n            | <b>next-intl 4</b>                                                       | Routes <code>/fr</code> <code>/de</code> <code>/en</code> , détection de langue, fallback sans clés visibles                                                                                                                      |
| Validation      | <b>Zod 4</b> (client + serveur, schéma unique)                           | Une seule source de vérité pour les règles du formulaire                                                                                                                                                                          |
| E-mails         | <b>Nodemailer</b> (SMTP au choix)                                        | Aucun fournisseur imposé, migration facile                                                                                                                                                                                        |
| Tests           | <b>Vitest</b> (unitaires) + <b>Playwright</b> (E2E)                      | Couverture des parcours critiques dans les trois langues                                                                                                                                                                          |
| Images          | <b>sharp</b> + <code>next/image</code> (AVIF/WebP)                       | Compression automatique, formats modernes                                                                                                                                                                                         |

Aucune dépendance propriétaire, aucun service payant obligatoire. Voir `docs/DEPENDANCES.md` (Dependances\_et\_couts.pdf) pour la liste complète, les licences et les coûts récurrents éventuels.

## 2. Démarrage rapide (développement local)

Prérequis : **Node.js ≥ 20.9**, **Docker** (pour PostgreSQL et MailHog).

```
git clone <url-du-depot> romial-kenmogne
cd romial-kenmogne
npm install
cp .env.example .env          # puis renseigner PAYLOAD_SECRET
docker compose up -d          # PostgreSQL :5437 + MailHog :8026
npm run seed                   # contenu de démarrage + compte administrateur
npm run dev                    # http://localhost:4313
```

| Adresse                                                               | Contenu                            |
|-----------------------------------------------------------------------|------------------------------------|
| <a href="http://localhost:4313/fr">http://localhost:4313/fr</a>       | Site public (français)             |
| <a href="http://localhost:4313/admin">http://localhost:4313/admin</a> | Administration CMS                 |
| <a href="http://localhost:8026">http://localhost:8026</a>             | MailHog — e-mails envoyés en local |

Guide détaillé : `docs/INSTALLATION.md` (Installation.pdf).

### 3. Scripts npm

| Script                                     | Rôle                                                                                                                                                                |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>npm run dev</code>                   | Serveur de développement (port 4313)                                                                                                                                |
| <code>npm run build</code>                 | Build de production                                                                                                                                                 |
| <code>npm start</code>                     | Serveur de production                                                                                                                                               |
| <code>npm run seed</code>                  | Injecte le contenu de démarrage et crée le premier administrateur                                                                                                   |
| <code>npm run import:assets</code>         | Importe les photos et la couverture fournies par le client (voir <code>docs/INSTALLATION.md</code> )                                                                |
| <code>npm run docs:pdf</code>              | Régénère tous les documents en PDF dans <code>docs/pdf/</code> (guide, éléments à fournir, présentation, installation, déploiement, CMS, dépendances, tests, audit) |
| <code>npm run brand:assets</code>          | Régénère le logo en PNG (icônes, image de partage, <code>public/brand/</code> ) à partir de <code>src/lib/brand-mark.ts</code> , avec la police Playfair Display    |
| <code>npm run docs:screenshots</code>      | Refait les captures du site public du guide ( <code>docs/images/guide/</code> ) sur un site démarré, avant <code>docs:pdf</code>                                    |
| <code>npm run generate:types</code>        | Régénère <code>src/payload-types.ts</code> après modification du modèle CMS                                                                                         |
| <code>npm run generate:importmap</code>    | Régénère la carte d'import de l'admin Payload                                                                                                                       |
| <code>npm run typecheck</code>             | Vérification TypeScript stricte                                                                                                                                     |
| <code>npm run lint</code>                  | ESLint (config Next.js 16)                                                                                                                                          |
| <code>npm run format / format:check</code> | Prettier                                                                                                                                                            |
| <code>npm test</code>                      | Tests unitaires (Vitest)                                                                                                                                            |
| <code>npm run test:e2e</code>              | Tests end-to-end (Playwright)                                                                                                                                       |
| <code>npm run verify</code>                | format + types + lint + tests unitaires                                                                                                                             |
| <code>npm run db:up / db:down</code>       | Démarre / arrête PostgreSQL et MailHog                                                                                                                              |

## 4. Structure du projet

```
src/
├─ app/
│   ├── (frontend)/[locale]/      Site public : une route par page, 3 langues
│   │   ├── page.tsx              Accueil
│   │   ├── about/ expertise/ experience/ insights/ books/ businesses/
│   │   ├── contact/ legal/[slug]/
│   │   ├── not-found.tsx error.tsx 404 d'une fiche inexistante, erreurs
│   │   └── globals.css            Design tokens + thème clair/sombre
│   ├── global-not-found.tsx        404 localisée des adresses inconnues (rendue côté serveur)
│   ├── (payload)/                  Administration CMS (généré par Payload)
│   ├── api/contact/route.ts        Endpoint du formulaire
│   ├── sitemap.ts robots.ts
│   └─ components/
│       ├── layout/ (Header, Footer) theme/ (thème clair-sombre)
│       ├── i18n/ (sélecteur FR|DE|EN) ui/ (primitives réutilisables)
│       ├── cards/ home/ insights/ experience/ contact/ books/
│       ├── seo/ (JSON-LD)           analytics/
│   └─ content/starter.ts            Contenu de démarrage (marqué « exemple »)
├─ i18n/                             Configuration des locales et de la navigation
├─ lib/                               cms, seo, email, format, validation, env, theme (apparence)
├─ messages/{en,fr,de}.json          Toutes les chaînes d'interface
├─ payload/                           Collections, globals, champs, accès, composants admin
├─ payload.config.ts                  Configuration du CMS
├─ proxy.ts                           Négociation de langue (ex-middleware)
├─ scripts/seed.ts                    Script d'amorçage
├─ tests/
│   ├── unit/                         Vitest (traductions, contrastes, apparence, validation, SEO)
│   ├── e2e/                          Playwright (navigation, i18n, thème, formulaire, contraste axe...)
│   └─ visual/capture.mjs              Captures d'écran pour la recette visuelle
└─ docs/                             Installation, déploiement, guide admin, checklists
```

## 5. Multilingue

- Langues : **français, allemand, anglais**, toutes disponibles au lancement.
- Langue par défaut documentée : **anglais** (`src/i18n/routing.ts`), cohérente avec le positionnement international ; la langue du navigateur est détectée et le visiteur est redirigé vers `/fr`, `/de` ou `/en`.
- Toutes les URL portent leur préfixe de langue (`/fr/expertise`).
- Les segments de route restent identiques dans les trois langues ; **les slugs des contenus CMS sont traduits** (`/fr/insights/lire-un-bilan-sans-etre-comptable` ↔ `/de/insights/eine-bilanz-lesen-ohne-buchhalter-zu-sein`).
- Le sélecteur `FR | DE | EN` s'appuie sur les balises `hreflang` de la page : changer de langue amène toujours sur **la page équivalente**.
- Chaque langue est précédée d'un **petit drapeau** dessiné en SVG (France, Allemagne, Royaume-Uni). Le drapeau ne remplace jamais le code de langue : un drapeau désigne un pays, pas une langue, et il est masqué aux lecteurs d'écran. Les drapeaux emoji ne sont pas utilisés (Windows ne les affiche pas).
- Le choix de langue est conservé dans un cookie (`RK_LOCALE`, 1 an).
- Une clé manquante en FR/DE retombe sur l'anglais ; **aucune clé technique n'est jamais affichée** (voir `src/i18n/request.ts`).

Pour ajouter une chaîne d'interface : l'ajouter dans `src/messages/en.json`, puis dans `fr.json` et `de.json`. Le test `tests/unit/messages.test.ts` échoue si une traduction manque, si une clé est en trop ou si un paramètre ICU diverge.

## 6. Thème clair / sombre

---

- Trois états : **clair**, **sombre**, **système** (par défaut).
- Le choix est mémorisé (`localStorage`), appliqué **avant le premier rendu** par un script bloquant : aucun flash de thème.
- Sans JavaScript, `prefers-color-scheme` s'applique quand même.
- Le thème sombre est une déclinaison dessinée (bleu nuit `#0b1728`, surfaces plus claires, or adouci), **pas une inversion automatique**.
- Tous les contrastes sont vérifiés automatiquement (`tests/unit/contrast.test.ts`, WCAG 2.2 AA sur les deux thèmes).
- **Apparence modifiable dans le CMS** (Administration → Apparence) : 6 palettes ou couleurs personnalisées pour les deux thèmes, police des titres, fond de la page d'accueil (halo, uni, image voilée). `src/lib/theme.ts` dérive tous les tokens et **ajuste les couleurs de texte pour rester en WCAG AA** quoi que l'administrateur choisisse (testé sur 300 palettes aléatoires + `tests/e2e/contrast.spec.ts` avec axe-core).

## 7. Supervision et données personnelles

---

- `GET /api/health` : `200` si le site et sa base répondent, `503` sinon — pour la sonde de disponibilité et le healthcheck Docker.
- Les demandes de contact inchangées depuis `CONTACT_RETENTION_MONTHS` mois (24 par défaut) sont supprimées chaque jour par le serveur (`src/instrumentation.ts`) ; `npm run purge:contacts` le fait à la demande.
- Mesure d'audience sans cookie (Umami / Plausible) ou **Google Analytics 4** avec bannière de consentement FR/DE/EN (rien n'est chargé avant l'accord du visiteur) : liste des événements dans `docs/GUIDE_ADMIN.md`, section 16.

## 8. Publication, aperçu et mise en cache

---

- Les pages publiques utilisent la **revalidation incrémentale** : elles sont servies depuis le cache et régénérées en arrière-plan toutes les **5 minutes**, avec des temps de réponse de l'ordre de 10 à 30 ms. Dès qu'un contenu public est publié, modifié ou supprimé dans le CMS, des hooks (`src/payload/hooks/revalidate.ts`) invalident le cache : la modification apparaît en quelques secondes, sans redéploiement. Un brouillon jamais publié ne déclenche rien.
- Le bouton **Preview** du CMS ouvre `/api/preview`, qui vérifie la session Payload (rôle administrateur ou éditeur) avant d'activer le mode brouillon de Next.js. La page affiche alors la version non publiée avec un bandeau et un bouton « Quitter l'aperçu ». Les aperçus ne sont ni mis en cache ni indexés. Le bouton existe pour les articles, livres, produits numériques, expertises, expériences, conférences & médias, pages de campagne, pages légales et activités (`src/payload/preview.ts`).
- L'**interface d'administration** est en français par défaut (allemand et anglais disponibles) : à ne pas confondre avec le sélecteur *Locale* de la barre supérieure, qui choisit la langue **du contenu** en cours d'édition.
- L'avatar du compte n'utilise pas Gravatar : aucune donnée (même hachée) n'est envoyée à un service tiers depuis l'administration.
- Les articles peuvent être **programmés** : une date de publication future les garde invisibles jusqu'à l'échéance.

## 9. Sécurité

---

- En-têtes : CSP, HSTS, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy` (`next.config.ts`).
- Formulaire : validation stricte client + serveur, honeypot, limitation de débit par IP, aucun contenu sensible dans les logs ou les URL.

- CMS : cookies `SameSite=Lax` + `Secure` en production, verrouillage après 5 tentatives, rôles Administrator / Editor, `/admin` en `noindex`.
- Le serveur **refuse de démarrer en production** si `PAYLOAD_SECRET` est absent, trop court ou laissé à sa valeur d'exemple.
- Aucun secret dans le dépôt : tout passe par `.env` (voir `.env.example`).

## 10. Documentation

| Document                                                           | Contenu                                                                       |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>docs/INSTALLATION.md</code> (Installation.pdf)               | Installation et lancement local pas à pas                                     |
| <code>docs/DEPLOIEMENT.md</code> (Deploiement_et_exploitation.pdf) | Staging, production, sauvegarde, restauration, rollback                       |
| <code>docs/GUIDE_ADMIN.md</code> (Guide_administrateur.pdf)        | Guide du propriétaire : publier, modifier, traduire                           |
| <code>docs/CMS.md</code> (Modele_de_contenu_CMS.pdf)               | Modèle de contenu détaillé (collections et champs)                            |
| <code>docs/ELEMENTS_A_FOURNIR.md</code> (Elements_a_fournir.pdf)   | Checklist des éléments encore attendus du commanditaire                       |
| <code>docs/RAPPORT_TESTS.md</code> (Rapport_de_tests.pdf)          | Rapport des tests exécutés et couverture                                      |
| <code>docs/AUDIT_MISE_EN_LIGNE.md</code> (Audit_mise_en_ligne.pdf) | Audit avant mise en ligne : bloquants, points importants, état de préparation |
| <code>docs/DEPENDANCES.md</code> (Dependances_et_couts.pdf)        | Dépendances, licences, coûts récurrents                                       |

## 11. État de la livraison

Ce que le site fait réellement aujourd'hui :

- ☒ 3 langues complètes, 2 thèmes, responsive 320 → 1440 px
- ☒ CMS opérationnel (contenus, traductions, médias, rôles, brouillons, programmation des articles)
- ☒ Formulaire de contact validé, anti-spam, e-mails FR/DE/EN **testés**
- ☒ SEO : métadonnées localisées, canonical, hreflang, sitemap, robots, données structurées Person / Article / Book / Organization / BreadcrumbList / WebSite
- ☒ Newsletter RK Insights intégrée : double opt-in, envoi des articles dans la langue de chaque abonné, désinscription en un clic, purge automatique
- ☒ Prise de rendez-vous : bouton vers l'outil externe (Cal.com, Calendly...) réglable dans le CMS, sans script tiers
- ☒ Page Conférences & médias (interventions à venir / passées, vidéos chargées au clic sans cookie, invitation préremplie)
- ☒ Synchronisation Google Sheets des demandes de contact et des abonnés confirmés **construite**, inactive tant que la clé du compte de service n'est pas configurée
- ☒ Export CSV des abonnés newsletter et des demandes de contact depuis l'administration (Excel, outil d'e-mailing, CRM)
- ☒ Qualification des prospects : questions facultatives, score sur 100 et priorité dans le CMS, priorité dans l'objet de la notification, rendez-vous proposé aux demandes prioritaires
- ☒ Médiathèque (vidéos, podcasts, interviews, presse) filtrable par format, thème et langue, alimentée par Conférences & médias
- ☒ Pages de campagne composées par blocs dans le CMS (en-tête, texte, points clés, vidéo, livres, produits, FAQ, appel à l'action, newsletter)
- ☒ Carte Europe–Afrique des expériences (calculée côté serveur, sans service tiers), filtre par pays accessible au clavier
- ☒ Apparence (palettes, couleurs, police, fond d'accueil) modifiable dans le CMS, contrastes AA garantis
- ☒ Déploiement Docker testé sur base vierge (migrations, seed, cache)

-  Point de santé `/api/health` et suppression automatique des demandes de contact expirées
-  Tests unitaires et E2E verts
-  Intégration continue GitHub Actions (`.github/workflows/ci.yml`) : formatage, TypeScript, lint, tests unitaires et build de production à chaque push sur `main` et à chaque pull request, plus les tests end-to-end (Chromium et mobile) sur une base PostgreSQL et un MailHog jetables remplis avec le contenu de départ ; Firefox et WebKit restent lancés en local
-  Image Docker de production construite après chaque CI verte (`.github/workflows/docker.yml`), publiée sur GHCR puis **déployée automatiquement** sur le serveur (sauvegarde avant mise à jour, retour à l'image précédente si le contrôle de santé échoue)
-  **En ligne** sur <https://romialkenmogne.com> (VPS, Apache, HTTPS Let's Encrypt), e-mails envoyés par SMTP Titan
-  Diaporama de photos dans le hero de la page d'accueil (réglable dans le CMS, pause et commandes au clavier, sans défilement si l'utilisateur limite les animations)
-  Pastilles de notification dans le menu de l'administration : nouvelles demandes de contact et nouveaux abonnés newsletter

Ce qui est **volontairement** inactif, faute d'éléments ou de prestataire :

-  Vente directe de livres et de produits numériques **construite** (panier, Stripe, PayPal, commandes, e-mails, espace membres avec connexion par lien e-mail, téléchargements protégés, formations avec progression) mais **fermée** tant que les clés de paiement ne sont pas configurées et que la boutique n'est pas ouverte dans le CMS — le site l'annonce clairement
-  Analytics (aucun outil imposé ; activation par variables d'environnement)
-  Contenus marqués « Contenu d'exemple » et pages légales entre crochets à remplacer par des informations validées avant d'annoncer le site — voir `docs/AUDIT_MISE_EN_LIGNE.md` et `docs/ELEMENTS_A_FOURNIR.md`